

Memory Management

2026-03-10

1s2026

Gabriela Bittencourt, João Pedro Leôncio and Vinícius Peixoto

Contents



1	Understanding Resources	4
1.a	Memory x Storage	5
1.b	Program x Process	6
1.c	Memory Management Historical Advance	7
1.d	Non-contiguous Memory Allocation	8
2	Memory management overview	9
2.a	Physical vs. virtual address spaces	10
2.b	Some terminology	13
3	Physical memory allocation	14
3.a	Physical memory allocation	15
4	Virtual memory	17
4.a	Virtual memory	18
4.b	Page tables	19
4.c	Walking the page tables	22
5	Bitwise idioms	23
5.a	Basics	24

Contents (ii)



5.b	Bitmasks	27
6	Configuring virtual memory and paging	34
6.a	Common idioms	35
7	Lab: Memory management	37
7.a	files	38
8	Free Pages	40
8.a	Linked list	41
8.b	Data Structs options for the <code>freelist</code>	42
9	Allocating	43
9.a	sessions	44
9.b	Attention when implementing	45
10	References	46
10.a	References	47

1 Understanding Resources

Memory x Storage



When we talk about ‘memory’ in OS context, we are talking about **RAM**

Memory is NOT Storage

- Memory
 - fast, expensive, volatile
- Storage
 - slow, cheap, consistent

Program x Process



- Program
 - Code. Static entity.
 - Needs storage
 - Example: Your C code from MC202: the .c file
- Process
 - The execution of the code. Dynamic entity.
 - Needs memory: the instructions will be load in specific space of memory (), variables will be set to specific spaces in memory ()
 - Example: The instance of your text editor while you are writing a program; the instance of the terminal running when you are navigating through it to compile your code and when you run it; the instance of your program during the short period of time your computer is executing your code

Memory Management Historical Advance



- Contiguous Memory Allocation (no abstraction of memory)
 - ▶ One part is reserved to the OS, the other runs a single process
 - ▶ One process at the time: in order to change the process executed, the OS must save the previous process in disk and then load the next one.
 - ▶ Simple memory management
 - ▶ Poor memory utilization
- Non-contiguous Memory Allocation
 - ▶ Allows each process to “see” the entire address space as its own
 - ▶ Multiple processes can coexist
 - ▶ Memory allocation logic is opaque to processes – the kernel handles virtual memory

Non-contiguous Memory Allocation

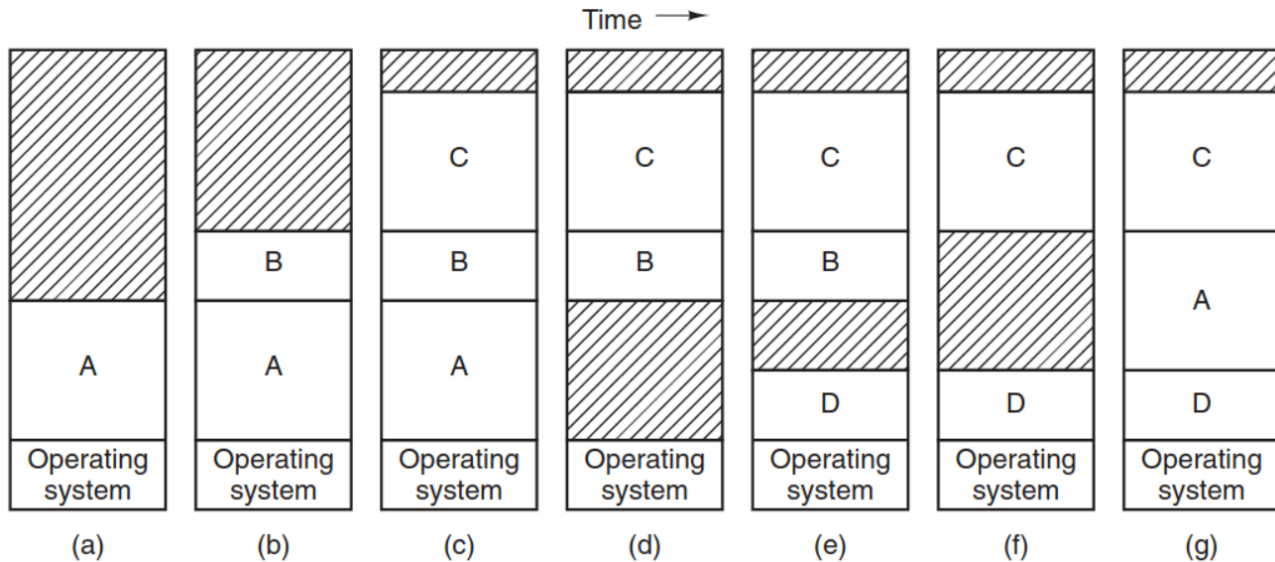


Figure 1: Memory allocation of 5 processes (programs: A, B, C, D and A again) over time.

2 Memory management overview

Physical vs. virtual address spaces



- **Physical addresses:** actual addresses as seen by the CPU on the system memory bus (e.g. DDR RAM, memory-mapped PCIe registers, etc.)
- **Virtual addresses:** made-up memory addresses that are mapped onto physical memory addresses

Example: `vmlinux` (a Linux kernel image)

```
$ llvm-objdump -t ~/kernel/linux/vmlinux
/home/nuke/kernel/linux/vmlinux:      file format elf64-littleaarch64
SYMBOL TABLE:
ffff80008228b738 l      .init.text      0000000000000054 record_mmu_state
ffff80008228b790 l      .init.text      0000000000000030 preserve_boot_args
ffff8000822655c8 l      F .rodata.text 0000000000000034 __primary_switch
ffff80008228b738 l      .init.text      0000000000000000 $x
ffff80008228b7c0 l      F .init.text      000000000000008c __primary_switched
ffff800080061230 l      F .text      0000000000000020 set_cpu_boot_mode_flag
```

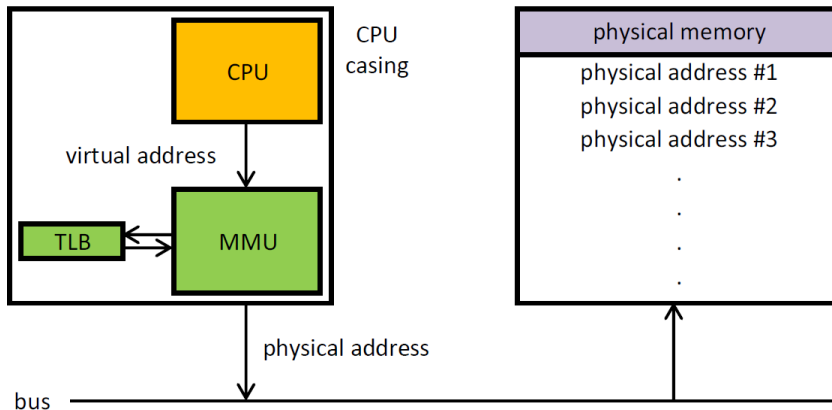
Note that the addresses for the symbols hover around address `0xffff8000822800` $\approx$ 65535 TiB (!)

Physical vs. virtual address spaces (ii)



- Clearly no system has that much memory
 - Might need to revisit this claim in the next decade or so :-)
- Those are **virtual addresses** configured by the kernel and translated into physical addresses
- This translation is done by a hardware unit called the Memory Management Unit (MMU)
 - The MMU is in turn configured by setting up data structures called **page tables**
 - Page tables describe the mapping between the virtual ↔ physical address spaces

Physical vs. virtual address spaces (iii)



CPU: Central Processing Unit

MMU: Memory Management Unit

TLB: Translation lookaside buffer

Figure 2: Overview of the address translation process.

Some terminology



- A **page** is a **block of memory** of fixed size
 - All memory is managed in units of pages
 - Traditionally 4KiB, more recently also 16KiB and 64KiB
 - In this class we always assume `PAGE_SIZE = 4096`
- Physical page number (**PPN**): the number of the n-th page in physical memory
 - Linux also calls this the **PFN** (page frame number)
 - Small exercise: what's the physical address of PPN=1024?
 - Another exercise: what's the PPN of physical address `0xffffffffabcde000`?
- Some commonly used idioms:
 - `PAGE_SHIFT`: number of bits to right-shift in order to get the PPN
 - For 4K pages, `PAGE_SHIFT = 12` ($2^{12} = 4096$)
 - `PAGE_SIZE`: the size of a page in bytes
 - `PAGE_SIZE = (1 << PAGE_SHIFT)`

3 Physical memory allocation

Physical memory allocation

Doing pretty much anything requires allocating physical memory. One of the first things a kernel needs to implement is a **physical page allocator**:

- Responsible for managing the available physical memory pages in the system
- Needs to keep track of which pages are in use and which ones are free
- Dumb (but perfectly valid) solution: freelist allocator
 - ▶ Linked list of all available pages
 - ▶ `alloc()` → pop the list head
 - ▶ `free(page)` → insert `page` at the front of the list

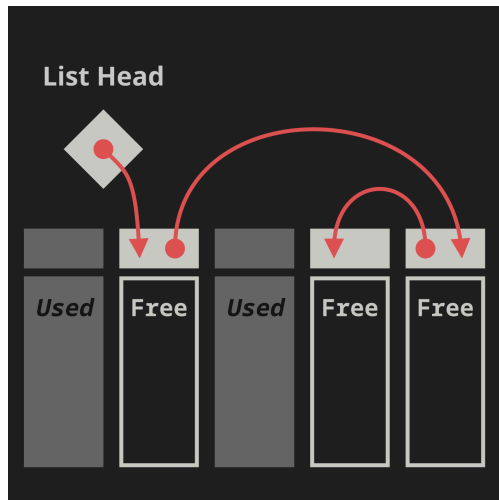


Figure 3: The freelist allocator (a linked-list of free pages)

Physical memory allocation (ii)



- This allocator has the upside of being dead simple to implement/debug
- But it would be nice to be able to allocate multiple contiguous pages at once
- In the next class we will go over other more elaborate allocators that solve this problem

4 Virtual memory

Virtual memory



- `riscv64` has 3 virtual memory modes (Sv39, Sv48 and Sv57)
 - We will work exclusively with Sv39 in this class
- We manage virtual memory by mapping **virtual page numbers** (VPNs) to **physical address numbers** (PPNs):

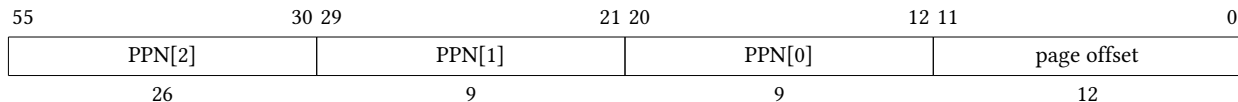


Figure 4: The Sv39 physical address encoding.

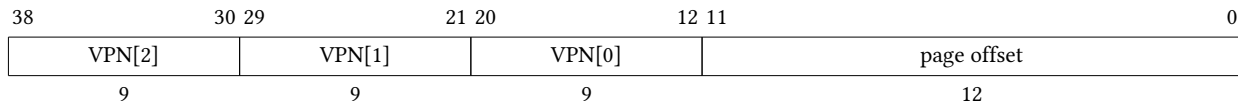


Figure 5: The Sv39 virtual address encoding.

Page tables



- In Sv39, a **page table** (PTB) is an array of $2^9 = 512$ words called **page table entries (PTEs)**
 - Note that in `riscv64` 1 word = 64 bits = 8 bytes and that $512 \times 8 \text{ bytes} = 4\text{KB}$
 - e.g. each page table takes up a physical page (not a coincidence)
- Each PTE can be either a:
 - **leaf page**: Map a VPN to a PPN
 - **non-leaf page**: Point to the PPN of another page table
- Sv39 uses a **3-level page table** scheme, which means that:
 - The **root page table** (also called a **level 1 page table**) contains entries that point to to the PPN of a **level 2 page table**
 - Each **level 2 page table** points to a **level 3 page table** containing only leaf pages that map
- The process of following entries from root PTB $\rightarrow$ level 2 PTB $\rightarrow$ level 3 PTB until we reach a leaf page is called **walking** the page tables

Let's take a closer look at the structure of a PTE:

Page tables (ii)



63	62	61	60	54	53	28	27	19	18	10	9	8	7	6	5	4	3	2	1	0
N	PBMT	Reserved			PPN[2]		PPN[1]		PPN[0]		RSW		D	A	G	U	X	W	R	V
1	2	7			26		9		9		2		1	1	1	1	1	1	1	1

Figure 6: Sv39 page table entry.

The X/W/R bits have the following meaning:

X	W	R	meaning
0	0	0	Pointer to a next level page table
0	0	1	Read-only page
0	1	0	Reserved for future use
0	1	1	Read-write page
1	0	0	Execute-only page
1	0	1	Read-execute page
0	1	0	Reserved for future use
1	1	1	Read-write-execute page

Page tables (iii)

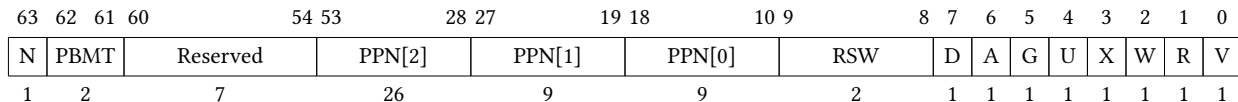
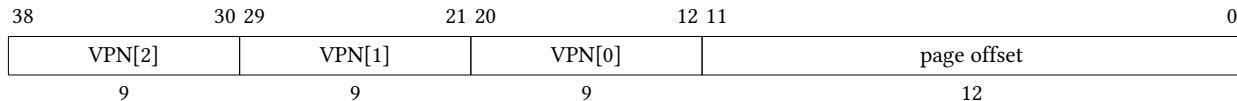


Figure 7: Sv39 page table entry.

For the others:

- **V** (valid): whether the PTE is valid
- **U** (user): whether the page is accessible in U-mode
- **G** (global): whether this is a global mapping that exists in all address spaces
- **A** (access): whether the page has been read/written/fetched since the bit was cleared
- **D** (dirty): whether the page has been written since the bit was cleared

Walking the page tables



We will go into more detail about how to actually implement this, but the (simplified) algorithm for walking the page tables is as follows:

- Start at the root page table (level 1)
- Check if the PTE at index `VPN[2]` has `V=1`; fail otherwise
- Move to the level 2 PTB at the physical address stored in `PPN[2:0]`
- Check if the PTE at index `VPN[2]` has `V=1`; fail otherwise
- Move to the level 1 PTB at the physical address stored in `PPN[2:0]`
- Extract the PPN that maps to the original VPN by looking at `PPN[2:0]`

5 Bitwise idioms

Basics



Bitwise operations: NOT ($\sim$), AND ($\&$), OR ($\mid$), XOR ($\wedge$)

A: 10110001

B: 01111001

$\sim$ A: 01001110

$\sim$ B: 10000110

A & B:

```
  10110001
& 01111001
-----
= 00110001
```

A $\mid$ B:

```
  10110001
| 01111001
-----
= 00000110
```

A $\wedge$ B:

```
  10110001
^ 01111001
-----
= 00000110
```


Basics (ii)



Left shift: <<

```
A:      1000000010110001
A << 5: 0001011000100000
```

n bit left-shift $\rightarrow$ multiply by 2^n

Logical right shift: >>

```
A:      1000000010110001
A >> 5: 0000010000000101
```

n bit logical right-shift $\rightarrow$ (unsigned) divide by 2^n

Basics (iii)



Logical right shift: `>>`

```
A:      1000000010110001
A >> 5: 0000010000000101
```

n bit (logical) right-shift $\rightarrow$ (unsigned) divide by 2^n

Arithmetic right shift: `>>`

```
A:      1000000010110001
A >> 5: 1111110000000101
```

n bit (arithmetic) right-shift $\rightarrow$ (signed) divide by 2^n

Watch out: in C, unsigned $\rightarrow$ **logical** left-shift, signed $\rightarrow$ **arithmetic** left-shift

```
unsigned int A = 0b1000000010110001;
A >>= 5;      // 0b0000010000000101
```

```
int A =      0b1000000010110001;
A >>= 5;     // 0b1111110000000101
```

Bitmasks



A **bitmask** is a carefully constructed number that helps us **read/write specific bits** of another number.

- Quick terminology note: “**clear** a bit” means **make it 0**, “**set** a bit” means **make it 1**

Suppose we have have $A = 0b1000000010110001$, and want to **read bit 4**. We use the following trick:

Read $A[4]$: $A \& (1 \ll 4)$

$\hat{=}$

$A \quad 0b1000000010110001$

$MASK \quad \& \ 0b00000000000010000 \quad \text{--->} \ (1 \ll 4)$

$= \ 0b00000000000010000$

Bitmasks (ii)



Similarly, if we wanted to **set bit 6**:

Set A[6]: $A \mid (1 \ll 6)$:

$\hat{A}$

A 0b1000000010110001

MASK | 0b0000000001000000 ---> $(1 \ll 6)$

= 0b1000000011110001

Bitmasks (iii)



What about **clearing bit 5**?

Clear A[5]: $A \& \sim(1 \ll 5)$:

$\hat{A}$

A 0b1000000010110001

~MASK & 0b111111111011111 ---> $\sim(1 \ll 5)$

= 0b1000000010010001

Bitmasks (iv)



Notice the pattern:

- We have some number (or register) `A`
- Want to modify bit `n`? $\rightarrow \text{MASK} = (1 \ll n)$
- Set bit `n` $\rightarrow A \mid= \text{MASK}$
- Clear bit `n` $\rightarrow A \&= \sim \text{MASK}$

Now let's generalize this to **multiple bits**. How to **set bits 12, 3 and 2**?

Bitmasks (v)



Set A[12], A[3:2]: A |= MASK

MASK = ((1 << 12) | (1 << 3) | (1 << 2))

â€œâ€œâ€œ

A 0b1000000010110001

MASK | 0b000100000001100

= 0b1001000010111101

What about **clearing bits 15, 4 and 5**?

Bitmasks (vi)



```
Clear A[15], A[5:4]: A &= ~MASK
MASK = ((1 << 15) | (1 << 5) | (1 << 4))
      ^          ^ ^ ^
A      0b10000000010110001
MASK   & 0b0111111111001111
      -----
      = 0b0000000001000001
```

Last example (may be relevant for your next lab ð): how to write 0b1101 to **bits 4, 5, 6 and 7**?

Bitmasks (vii)



At this point we might as well do it in plain C:

```
#define MASK ((1UL << 7) | (1UL << 6) | (1UL << 5) | (1UL << 4))
#define SHIFT 4
uint64_t a = 0b1000000010110001
a &= ~MASK; // clear a[7:4]
a |= (0b1101UL << SHIFT)
```

Notice we use `1UL` for the constants in the macro; numeric constants in C **default to `int` (signed)**, but we generally want to **avoid signed bit arithmetic**; the `UL` suffix tells specifies that the numeric constant is an **unsigned long int** (e.g. 64-bit unsigned integer).

Always try to avoid mixing signed and unsigned numbers as it is one of the biggest sources of **undefined behavior** in C.

6 Configuring virtual memory and paging

Common idioms

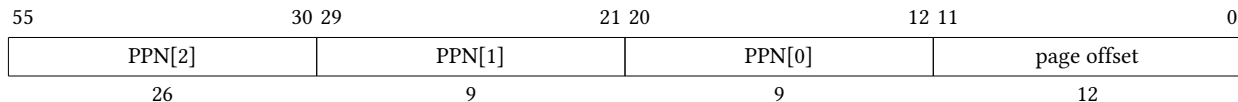


Figure 9: The Sv39 physical address encoding.

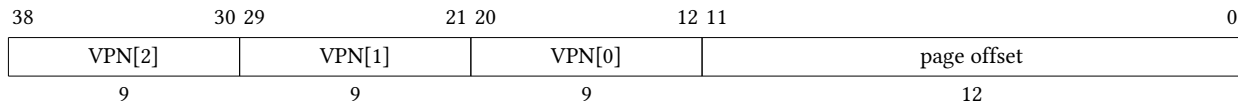


Figure 10: The Sv39 virtual address encoding.

```
#define PAGE_SHIFT 12
#define PAGE_SIZE (1 << PAGE_SHIFT)
```

Note:

1 << 12 = 0b10000000000000

Common idioms (ii)



index	PTE
0x00000000	0xffff



index	PTE
0x00000000	0xffff
0x00000000	0xffff

7 Lab: Memory management

files



```
|— include (kernel headers)
|   |— arch
|       |— pgtable.h (defs and helpers for page table)
|       |— kalloc.h (freelist implementation)
|— src
|   |— mm.c (implementation for memory management)
```

files (ii)



- `pgtable.h`: All the hardware specs and design for the page table
- `kalloc.h`: Implement a `freelist` – non-allocated pages of memory
 - `page_alloc()`: pop the data struct of freelist
 - `page_free()`: insert in data struct of freelist
- `mm.c`: Memory management
 - `ptb_walker`: walk through the levels of pagetable and returns when valid or !valid, or leaf or !leaf (it is up to implementation)
 - `vm_page_alloc`: allocate a page given a virtual address
 - `mm_init`: allocate all sessions: `.bss`, `.text`, `.data`, `.rodata`

8 Free Pages

Linked list



```
struct node {  
    int data;  
    struct node *next;  
}
```

To add new node:

```
struct create_node {  
    struct node new_node = (struct node*)malloc(sizeof(struct node)); // :(  
    // ...  
}
```

We cannot use a `malloc` when implementing the precursor of `malloc`.

We cannot dynamic alloc the memory if there is no memory management yet.

Data Structs options for the freelist



- Linked list, with all the nodes declared in array
- Directly an array with the addresses and a walker
- Bitmap the used pages and calculate the free addresses from the position of the bit
- Use the pages as node of the linked list – write at the first word, the address for next page

9 Allocating

sessions



which flag to use in each session? (PTE_READ , PTE_WRITE , PTE_EXEC)

- `.text`: contains executable instructions
- `.bss`: contains non-initialized statically allocated variables
- `.rodata`: contains read-only data
- `.data`: contains data that can be altered

Attention when implementing



```
*** Fatal Exception *** core dump ***  
Segmentation fault (core dumped)
```

Will never occur in your code.

10 References

References



- Tanenbaum, Andrew. (2012) '3. Memory Management', in *Modern operating systems*. Pearson Education, Inc., 4th ed.
- *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*. <https://riscv.github.io/riscv-is-manual/snapshot/privileged/>
- *Virtual Memory Layout on RISC-V Linux*. <https://docs.kernel.org/arch/riscv/vm-layout.html>